Questions of the day

- Graph distance is defined as the length of the shortest path between two vertices

- How do we compute graph distance?
- Do we need to look at all paths and find the shortest?
- What if there are a lot of paths?
- What if the edges are different lengths?

Graph traversals and weighted graph algorithms

William Hendrix

Lecture 10

Outline

- Review
  - Graph algorithm analysis
  - BFS
  - DFS
- Traversal complexity
- Traversal trees
- Applications of BFS/DFS
- Dijkstra's Algorithm
- Prim's Algorithm
- Kruskal's Algorithm
- Topological sorting

Graph algorithm analysis review

- May include graph features
  - n: # vertices
  - m: # edges
  - deg(v): degree of v
- Possibly consider graph operations separately

Operation	Adjacency matrix	Adjacency list	Edge list
Graph(n)	$\Theta(n^2)$	$\Theta(n)$	$\Theta(1)$
AddEdge(u, v)	$\Theta(1)$	$\Theta(1)$*	$\Theta(m)$
RemoveEdge(u, v)	$\Theta(1)$	$\Theta(1)$*	$\Theta(m)$
IsAdjacent(u, v)	$\Theta(1)$	$\Theta(1)$*	$\Theta(\lg(m))$
GetNeighbors(v)	$\Theta(n)$	$\Theta(\deg v)$	$\Theta(\mathrm{m})$
Space:	$\Theta(n^2)$	$\Theta(m+n)$	$\Theta(m)$

- Handshaking Lemma: $\sum_{v \in V} \deg v = 2m = \Theta(m)$
  - All neighbors of all vertices = all edges

Traversal review

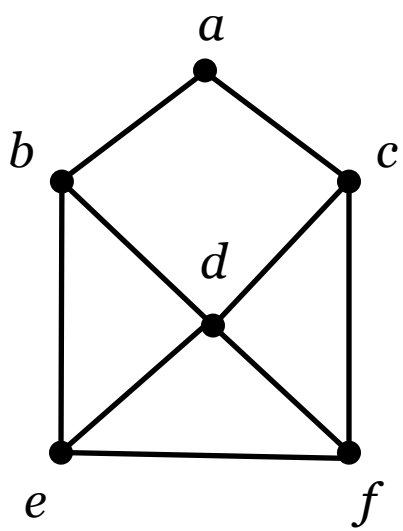
- BFS: queue-based traversal
  - Iteratively dequeue and add undiscovered neighbors
  - Discover when adding to queue, explore when removing
  - Outer loop finds next undiscovered node and restarts inner loop

- DFS: recursive (stack-based) traversal
  - Recursively explore each undiscovered neighbor
  - Discover before processing neighbors, explore afterwards
  - Outer function finds next undiscovered node and runs recursion

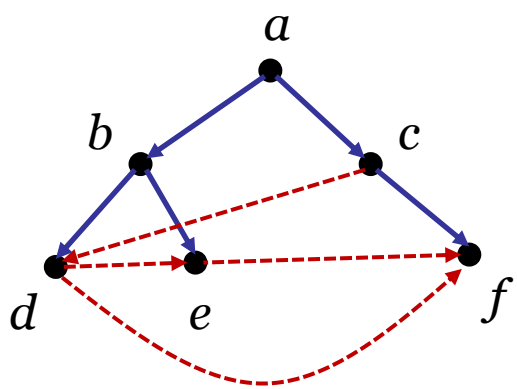
- Both explore vertices via edges
- Both mark vertices undiscovered, discovered, then explored
- Both are $\Theta(n + m)$ with adjacency list
  - Mark all vertices: $\Theta(n)$
  - Call getNeighbors on every node: $\Sigma \deg(v) = \Theta(m)$

BFS and DFS traversal trees

- Traversal tree
  - Set of edges traversed by BFS or DFS,
  - Augmented with other edges of graph

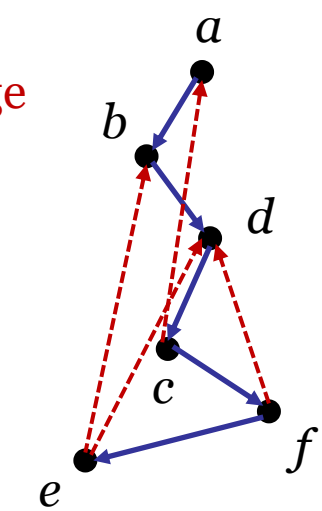


BFS



Tree edge
Non-tree edge

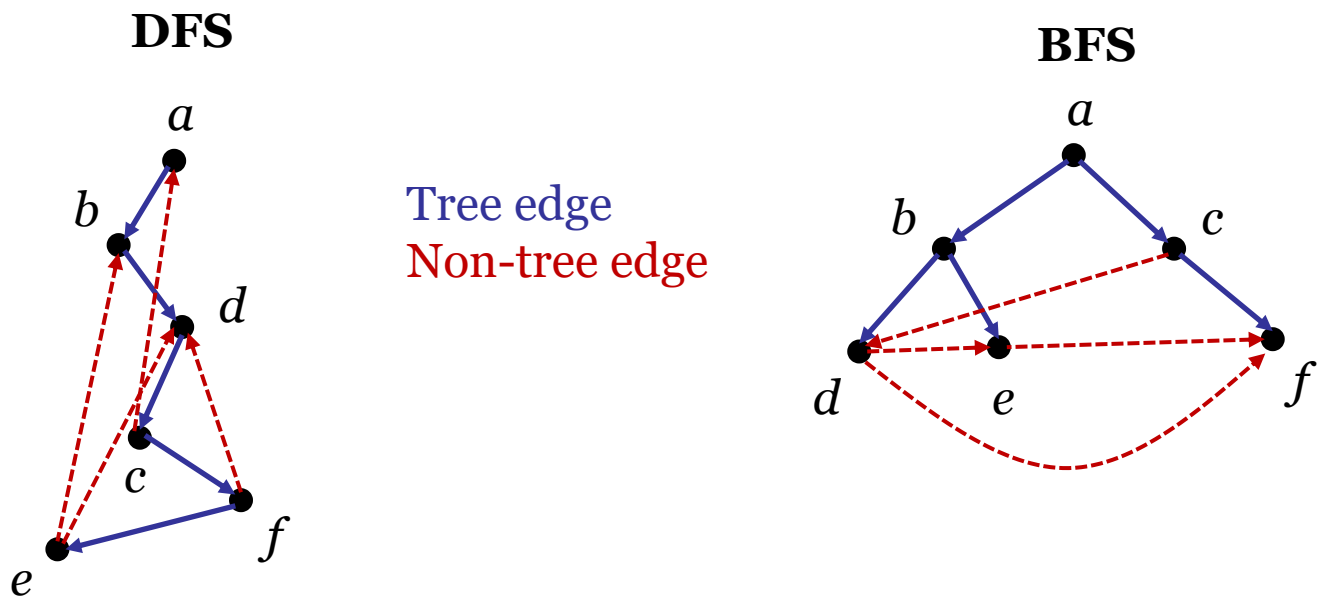
DFS



- Non-tree edges are siblings, cousins, or nieces/nephews
  - "Cross-edges"

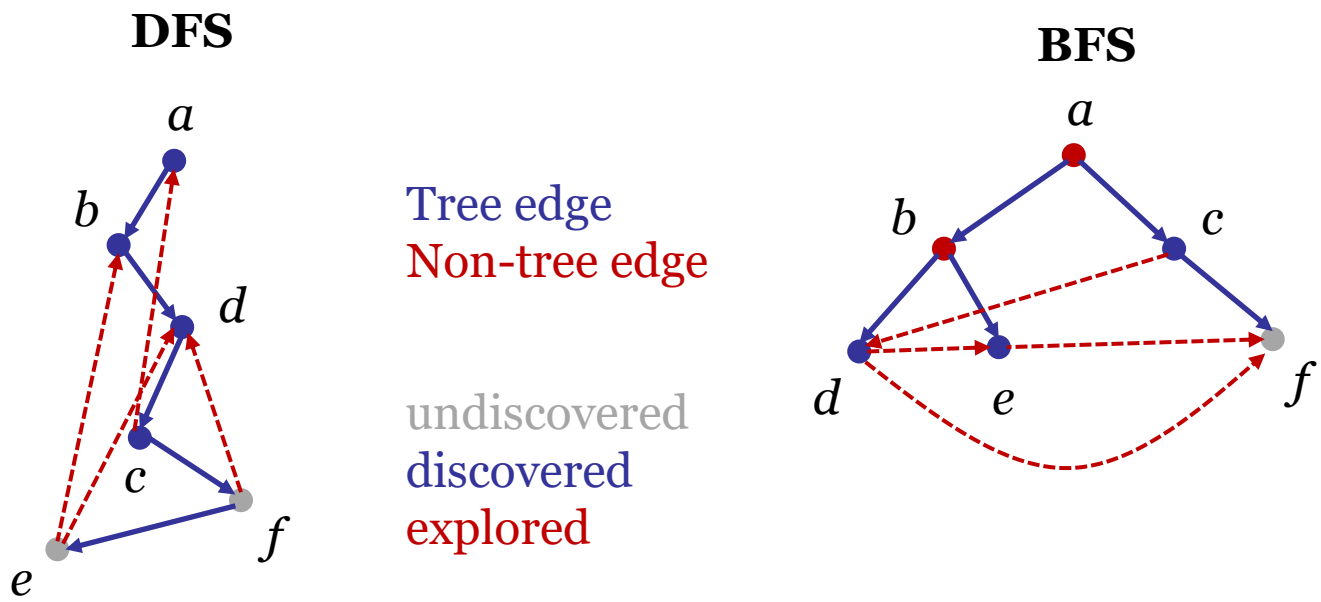
- Non-tree edges are ancestors
  - "Back-edges"

Traversal tree structure



- **Observation 1:** non-tree edges connect to discovered vertices
  - Tree edges are undiscovered
- **Observation 2:** vertices are discovered iff they are in stack/queue

Traversal tree structure



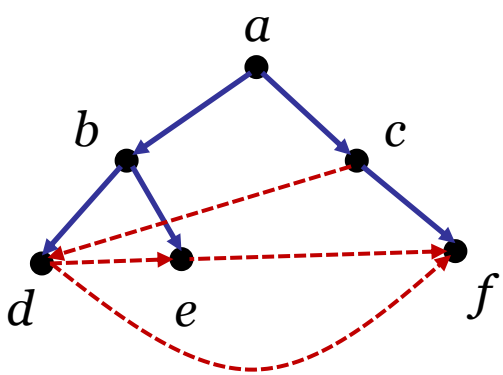
- **Observation 1:** non-tree edges connect to discovered vertices
  - Tree edges are undiscovered
- **Observation 2:** vertices are discovered iff they are in stack/queue
  - Stack: all vertices back to root
  - Queue: remaining vertices on same level + vertices on next level
- DFS: non-tree-edges are ancestors ("back edges")
- BFS: non-tree-edges are on same or next level ("cross edges")

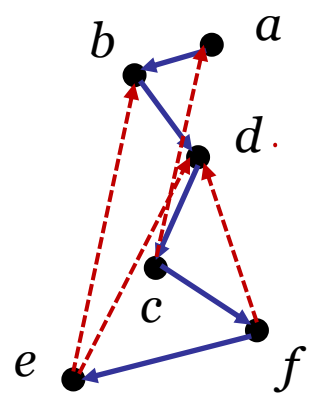
Graph traversal example

- *Recall:* length of the *shortest* path between two vertices
  - In undirected graph, fewest number of edges

1. Should we use BFS or DFS? Why?

- *Hint:* what is *d(a, e)* in the graph below?





2. Prove that *d(a, v)* equals the level of *v* in a BFS traversal tree starting at *a*, for every vertex *v*.

Graph traversal example

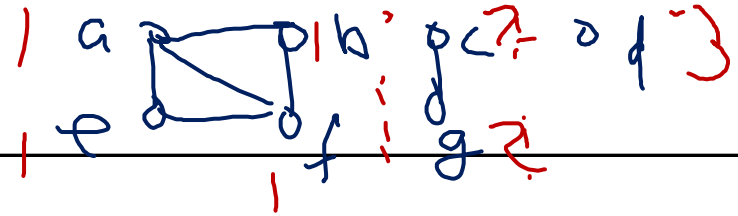
1. BFS, because *d(a, v)* equals the level of *v* in the BFS traversal tree starting at *a*
2. *Proof.* We use induction on n to prove that $d(a, v) = n$ if and only if v is on level n of the BFS traversal tree starting at a.

   (*Base case*) When v is on level 0 of the tree, $v = a$, and $d(a, a) = 0$. When $d(a, v) = 0$, $v = a$, so v must be on level 0 of the tree.

   (*Inductive step*) Suppose that $d(a, u) = k$ for every vertex u on level k of the tree and that every vertex with distance k to a is on level k, and suppose that vertex v is on level $k + 1$ of the tree. Since v is on level $k + 1$ of the tree, its parent p must be on level k. By the inductive hypothesis, $d(a, p) = k$, so there must be some path $P : a, v_1, v_2, \ldots, v_{k_1}, p$ of length k from a to p. The sequence $Q : a, v_1, v_2, \ldots, v_{k-1}, p, v$ must be a path from a to v of length $k + 1$, so $d(a, v) \leq k + 1$. Moreover, there can't be any shorter path, as v cannot have any neighbor higher than level k in the tree.

   Conversely, if a $d(a, v) = k+1$, then there is some path $P : a, v_1, v_2, \ldots, v_k, v$ of length $k+1$ from a to v. The second-to-last vertex on this path, v_k, must be distance k to a, so it is on level k. Since v is adjacent to a vertex on level k, it must be on levels $k - 1$ to $k + 1$, but it can't be on level $k - 1$ or k because these levels have distance $k - 1$ and k to a, respectively. Thus, v must be on level $k + 1$ of the BST traversal tree. $\square$

Traversal exercise



- Design an algorithm to label each vertex with a *component ID* such that all vertices in the same connected component have the same ID.
  - What is the runtime of your algorithm when using an adjacency list?

```
BFS:
Mark v_0 discovered
Enqueue v_0
id = 1
While queue not empty:
    Dequeue vertex v
    Mark v explored
    compID[v] = id
    Mark children discovered
    Enqueue children
id++
If any vertex still undiscovered:
    Choose one and repeat
```

```
DFS:
id = 1
While some vertices undiscovered:
    v = next undiscovered vertex
    DFS-Rec(v)
    id++

DFS-Rec(v):
Mark v as discovered
compID[v] = id
For undiscovered neighbors u:
    DFS-Rec(u)
Mark v explored
```

Traversal exercise

- Design an algorithm to label each vertex with a *component ID* such that all vertices in the same connected component have the same ID.
  - What is the runtime of your algorithm when using an adjacency list?

- **ComponentDFS:**

  components = 0

  While vertices still undiscovered

  components = components + 1

  DFS-Rec(v) //label all vertices with ID components

  Return components

- Can also be done w/ BFS
  - Increment components every time you "repeat" (last line)
- Complexity: $\Theta(n + m)$

BFS vs. DFS

Both BFS and DFS:

- Traverse one component at a time
- Take $\Theta(n+m)$ time
- Traversal trees have special properties
  - Sometimes critical, sometimes unimportant

- DFS is somewhat easier to implement
- Usage depends on application

BFS and DFS application examples

Both:

- Component detection
- Cycle detection

BFS:

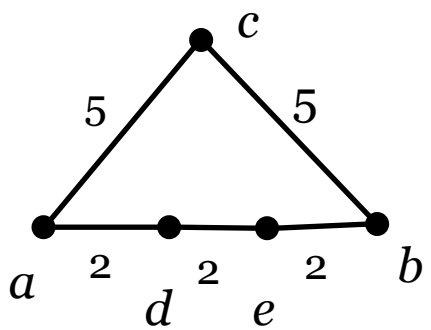
- Unweighted distance
- Radius and center of unweighted graph
- Betweenness centrality
  - Ranks vertices by “importance”
  - Counts shortest paths that pass through a vertex

DFS:

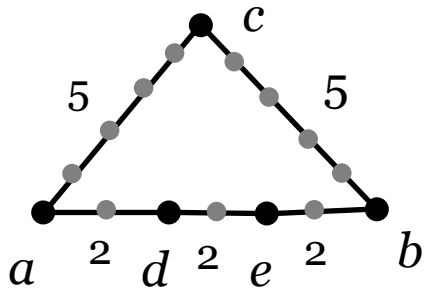
- Cut edge or cut vertex detection
- Sorting (on BST)
- Topological sorting
  - Order vertices of Directed Acyclic Graph (DAG) so all edges point to right
- *Backtracking*

Weighted graph distance

- Edge weights may represent distance or strength of connection
  - E.g., road network or chemical interactions
- Distance: sum of edge weights
- BFS doesn't necessarily find shortest path:



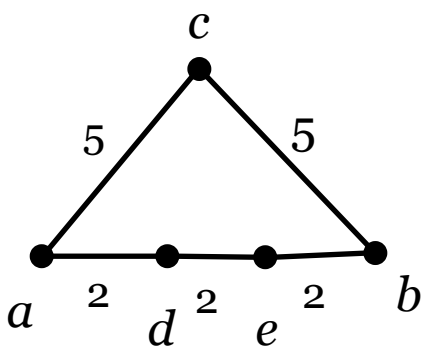
  - Finds path with fewest edges
- Naïve strategy: subdivide edges so that all edges have same length



- Finds shortest path
- Complexity depends on edge weights and GCD

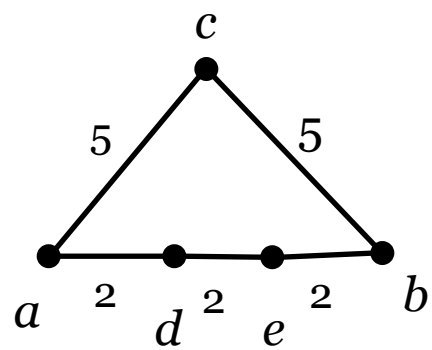
A better solution

- Don't actually add "interstitial" vertices
- Process vertices in same order as naïve strategy
  - Store when we would reach each vertex



A better solution

- Don't actually add "interstitial" vertices
- Process vertices in same order as naïve strategy
  - Store when we would reach each vertex



- t=0: *a*
  - Reach *c* at 5, *d* at 2
- t=2: *d*
  - Reach *e* at 4
- t=4: *e*
  - Reach *b* at 6
- t=5: c
  - Reach *b* at 10 (worse than projected)
- t=6: *b*

Dijkstra's algorithm

- Pseudocode

```
Input: G = (V, E): graph in question
Input: a, b: vertices to measure distance between
Input: n, m: order and size of G
Label all vertices as unreachable except a
v = a, D = 0
while v ≠ b do
    for u ∈ N(v) do
        Label u with distance D + d(v, u) unless u has a smaller label
    end
    Choose reachable vertex v with min distance D
end
return D
```

  - To return the path, add a back-link every time you label *u*
  - Traverse back-links from *b* to *a*

- Greedy algorithm
  - Always selects vertex with min distance

Dijkstra's analysis

```
Input: G = (V, E): graph in question
Input: a, b: vertices to measure distance between
Input: n, m: order and size of G
Label all vertices as unreachable except a
v = a, D = 0
while v ≠ b do
    for u ∈ N(v) do
        Label u with distance D + d(v, u) unless u has a smaller label
    end
    Choose reachable vertex v with min distance D
end
return D
```

Dijkstra's analysis

Complexity	
	Input: $G = (V, E)$: graph in question
	Input: a, b: vertices to measure distance between
	Input: n, m: order and size of G
$\Theta(n)$	Label all vertices as unreachable except a
$\Theta(1)$	$v = a$, $D = 0$
	while $v \neq b$ **do**
$\Theta(\deg(v))$	**for** $u \in N(v)$ **do**
$\Theta(1)^*$	Label u with distance $D + d(v, u)$ unless u has a smaller label
	end
	Choose reachable vertex v with min distance D
	end
$\Theta(1)$	**return** D

Dijkstra's analysis

Complexity

$\Theta(n)$

$\Theta(\deg(v))^*$

Depends

$\Theta(1)$

Input: $G = (V, E)$: graph in question
Input: a, b: vertices to measure distance between
Input: n, m: order and size of G

```
1 Label all vertices as unreachable except a
2 v = a, D = 0
3 while v ≠ b do
4     for u ∈ N(v) do
5         Label u with distance D + d(v, u) unless u has a smaller label
6     end
7     Choose reachable vertex v with min distance D
8 end
9 return D
```

- Line 7: depends on implementation
  - Also affects line 5
- Unsorted array:
  - Min is $\Theta(n)$
- Heap:
  - Min is $\Theta(\lg n)$
  - Insert is $\Theta(\lg n)$
  - Decrease is $\Theta(\lg n)$

Dijkstra's analysis

<u>Complexity</u>

$\Theta(n)$

$\Theta(\deg(v) \lg n)$

$\Theta(\lg n)$

$\Theta(1)$

Input: $G = (V, E)$: graph in question
Input: a, b: vertices to measure distance between
Input: n, m: order and size of G

```
Label all vertices as unreachable except a
v = a, D = 0
while v ≠ b do
    for u ∈ N(v) do
        Label u with distance D + d(v, u) unless u has a smaller label
    end
    Choose reachable vertex v with min distance D
end
return D
```

Dijkstra's analysis

<u>Complexity</u>

	Input: $G = (V, E)$: graph in question
	Input: a, b: vertices to measure distance between
	Input: n, m: order and size of G
$\Theta(n)$	Label all vertices as unreachable except a
	$v = a$, $D = 0$
$O(n)$ iters	**while** $v \neq b$ **do**
	for $u \in N(v)$ **do**
$\Theta(\deg(v) \lg n)$	Label u with distance $D + d(v, u)$ unless u has a smaller label
	end
	Choose reachable vertex v with min distance D
	end
$\Theta(1)$	**return** D

Dijkstra's analysis

Complexity

$\Theta(n)$

$O\left(\sum_{v \in V} \deg(v) \lg n\right)$
$= O(m \lg n)$

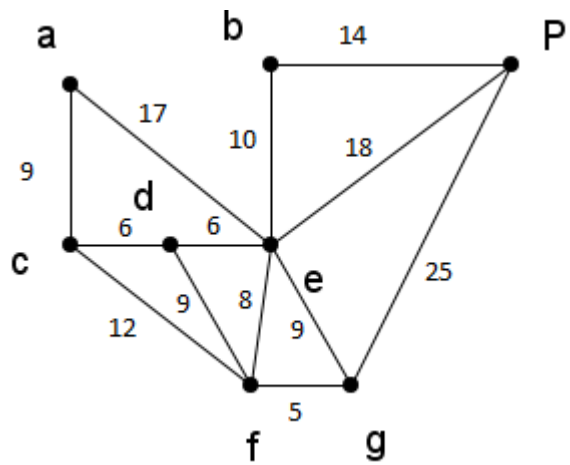
$\Theta(1)$

```
Input: G = (V, E): graph in question
Input: a, b: vertices to measure distance between
Input: n, m: order and size of G
Label all vertices as unreachable except a
v = a, D = 0
while v ≠ b do
    for u ∈ N(v) do
        Label u with distance D + d(v, u) unless u has a smaller label
    end
    Choose reachable vertex v with min distance D
end
return D
```

- Time complexity: $O(n + m \lg n)$
- Space complexity
  - Dominated by graph
  - $\Theta(n + m)$
- Same complexity to return (a, b)-path
  - Add parent links when you update labels
  - Follow links backwards from goal
- Same complexity to find distance to every other vertex

Minimum spanning tree

- Consider the following scenario:
  - Imagine you are a creating a new power company
  - Need to connect customers to your power plant
  - Need to minimize costs
- **Example**
  - Connect *P* to *a*-*g*:



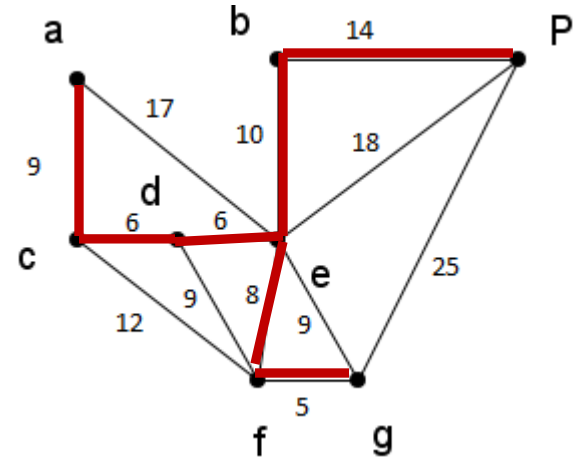
- Subgraph of a connected weighted graph that:
  - Connects all vertices of the graph (*spans*)
  - Has no cycles (*tree*)
  - Has the lowest edge weight sum among all spanning trees

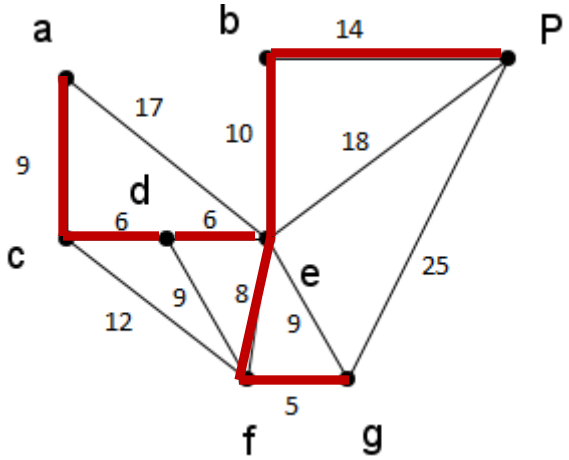
Computing the MST

- Two main algorithms
  - Both greedy

- Prim's algorithm
  - Starts at one vertex
  - Adds edge to nearest unconnected vertex
  - Repeat until spanning

- Kruskal's algorithm
  - Sort edges
  - Adds min edge between disconnected vertices
  - Repeat until connected





Prim's analysis

```
Input: G = (V, E): graph in which to find MST
Input: n, m: order and size of G
Output: Minimum spanning tree of G
edge = Array(n)
Let heap be a heap where every vertex has value ∞
Choose v_0 and decrease its value in heap to 0
for i = 1 to n do
    v = heap.DeleteMin()
    If v ≠ v_0, add (v, edge[v]) to MST
    for u ∈ N(v) do
        if u ∈ heap and d(u, v) < value(u) then
            Decrease value of u to d(u, v) in heap
            edge[u] = v
        end
    end
end
return MST
```

Prim's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum spanning tree of G
$\Theta(1)$	$edge = \text{Array}(n)$
$\Theta(n)$	Let $heap$ be a heap where every vertex has value ∞
$\Theta(\lg n)$	Choose v_0 and decrease its value in $heap$ to 0
	for $i = 1$ to n **do**
	$\quad v = heap.\text{DeleteMin}()$
	$\quad$ If $v \neq v_0$, add $(v, edge[v])$ to MST
	$\quad$ **for** $u \in N(v)$ **do**
$\Theta(1)$	$\quad\quad$ **if** $u \in heap$ and $d(u, v) < \text{value}(u)$ **then**
$\Theta(\lg n)$	$\quad\quad\quad$ Decrease value of u to $d(u, v)$ in $heap$
$\Theta(1)$	$\quad\quad\quad$ $\text{edge}[u] = v$
	$\quad\quad$ **end**
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Prim's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum spanning tree of G
$\Theta(1)$	$edge = \text{Array}(n)$
$\Theta(n)$	Let $heap$ be a heap where every vertex has value ∞
$\Theta(\lg n)$	Choose v_0 and decrease its value in $heap$ to 0
	for $i = 1$ to n **do**
	$\quad v = heap.\text{DeleteMin}()$
	$\quad$ If $v \neq v_0$, add $(v, edge[v])$ to MST
$\Theta(\deg(v))$ iters	$\quad$ **for** $u \in N(v)$ **do**
	$\quad\quad$ **if** $u \in heap$ and $d(u, v) < \text{value}(u)$ **then**
$\Theta(\lg n)$	$\quad\quad\quad$ Decrease value of u to $d(u, v)$ in $heap$
	$\quad\quad\quad$ $\text{edge}[u] = v$
	$\quad\quad$ **end**
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Prim's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum spanning tree of G
$\Theta(1)$	$edge = \text{Array}(n)$
$\Theta(n)$	Let $heap$ be a heap where every vertex has value ∞
$\Theta(\lg n)$	Choose v_0 and decrease its value in $heap$ to 0
$\Theta(n)$ iters	**for** $i = 1$ to n **do**
$\Theta(\lg n)$	$\quad v = heap.\text{DeleteMin}()$
$\Theta(1)$	$\quad$ If $v \neq v_0$, add $(v, edge[v])$ to MST
	$\quad$ **for** $u \in N(v)$ **do**
	$\quad\quad$ **if** $u \in heap$ and $d(u, v) < \text{value}(u)$ **then**
$\Theta(\deg(v) \lg n)$	$\quad\quad\quad$ Decrease value of u to $d(u, v)$ in $heap$
	$\quad\quad\quad$ $\text{edge}[u] = v$
	$\quad\quad$ **end**
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Prim's analysis

<u>Complexity</u>

Input: $G = (V, E)$: graph in which to find MST
Input: n, m: order and size of G
Output: Minimum spanning tree of G

$\Theta(1)$ — $edge = \text{Array}(n)$

$\Theta(n)$ — Let $heap$ be a heap where every vertex has value ∞

$\Theta(\lg n)$ — Choose v_0 and decrease its value in $heap$ to 0

$\Theta(m \lg n)$ —

```
for i = 1 to n do
    v = heap.DeleteMin()
    If v ≠ v_0, add (v, edge[v]) to MST
    for u ∈ N(v) do
        if u ∈ heap and d(u, v) < value(u) then
            Decrease value of u to d(u, v) in heap
            edge[u] = v
        end
    end
end
```

$\Theta(1)$ — **return** MST

Total: $\Theta(n + m \lg n)$

Kruskal's analysis

```
Input: G = (V, E): graph in which to find MST
Input: n, m: order and size of G
Output: Minimum Spanning Tree of G
E' = sort(E)
uf = UnionFind(n)
for i = 1 to m do
    (u, v) = E'[i]
    if uf.Find(u) ≠ uf.Find(v) then
        Add (u, v) to MST
        uf.Union(u, v)
    end
end
return MST
```

Kruskal's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum Spanning Tree of G
$\Theta(m \lg m)$	$E' = \text{sort}(E)$
$\Theta(n)$	$uf = \text{UnionFind}(n)$
	for $i = 1$ to m **do**
$\Theta(1)$	$(u, v) = E'[i]$
$\Theta(\alpha(n))$	**if** $uf.\text{Find}(u) \neq uf.\text{Find}(v)$ **then**
$\Theta(1)$	Add (u, v) to MST
$\Theta(\alpha(n))$	$uf.\text{Union}(u, v)$
	end
	end
$\Theta(1)$	**return** MST

Kruskal's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum Spanning Tree of G
$\Theta(m \lg m)$	$E' = \text{sort}(E)$
$\Theta(n)$	$uf = \text{UnionFind}(n)$
$\Theta(m)$	**for** $i = 1$ to m **do**
	$\quad (u, v) = E'[i]$
	$\quad$ **if** $uf.\text{Find}(u) \neq uf.\text{Find}(v)$ **then**
$\Theta(\alpha(n))$	$\quad\quad$ Add (u, v) to MST
	$\quad\quad uf.\text{Union}(u, v)$
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Kruskal's analysis

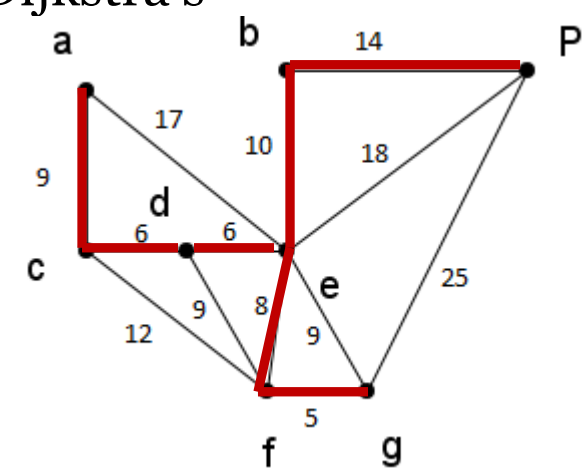
Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum Spanning Tree of G
$\Theta(m \lg m)$	$E' = \text{sort}(E)$
$\Theta(n)$	$uf = \text{UnionFind}(n)$
	for $i = 1$ to m **do**
	$\quad (u, v) = E'[i]$
	$\quad$ **if** $uf.\text{Find}(u) \neq uf.\text{Find}(v)$ **then**
$\Theta(m\alpha(n))$	$\quad\quad$ Add (u, v) to MST
	$\quad\quad uf.\text{Union}(u, v)$
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Total complexity: $\Theta(n + m \lg m)$

MST analysis

- Prim's vs. Kruskal's
  - Prim's: $\Theta(n + m \lg n)$
  - Kruskal's: $\Theta(n + m \lg m)$
  - Asymptotically equivalent
    - Prim's typically faster for dense graphs
    - Similar time for sparse graphs
  - Linear space for both

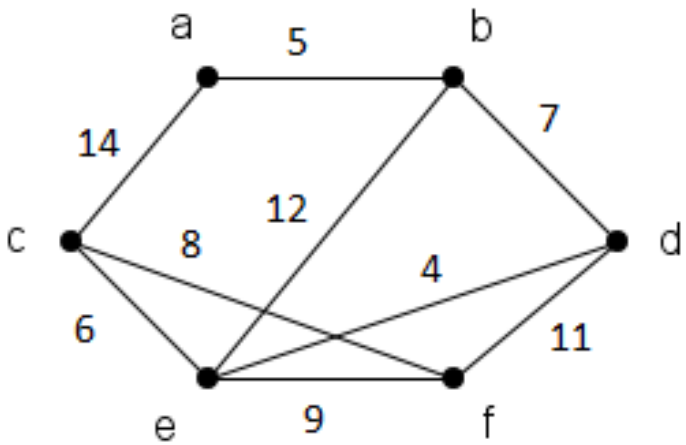
- Note: Prim's algorithm very similar to Dijkstra's
- BUT: MST ≠ min distance

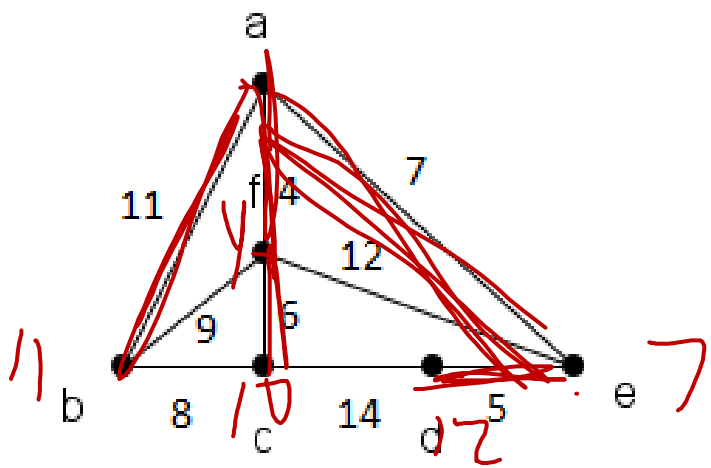


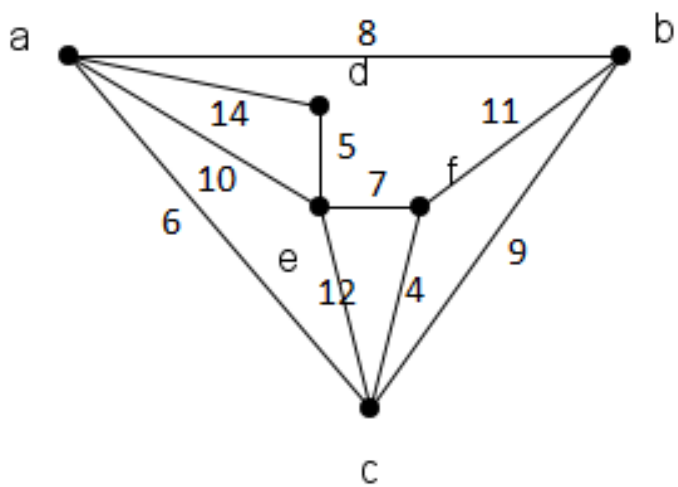
- MST Applications
  - Connecting spatial points
  - Travelling Salesman Problem
  - Clustering

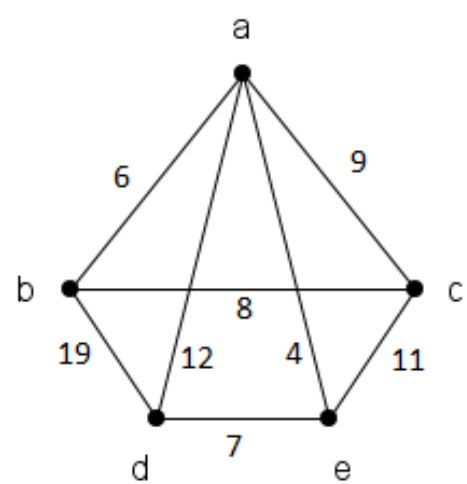
Graph algorithm exercise

- For each graph below, compute the tree produced by Prim's and Dijkstra's algorithms, starting at *a*



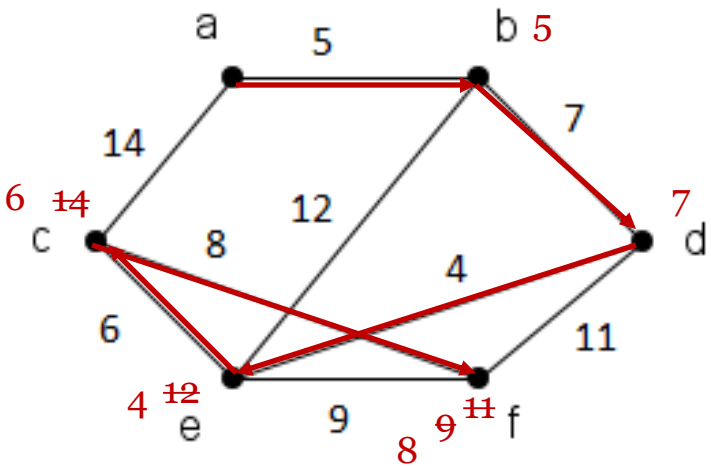


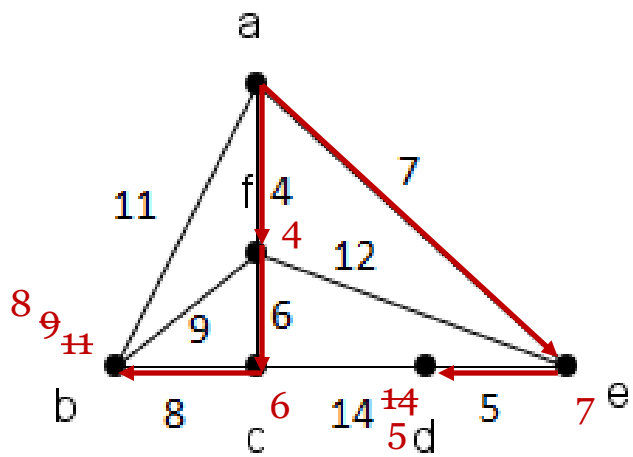


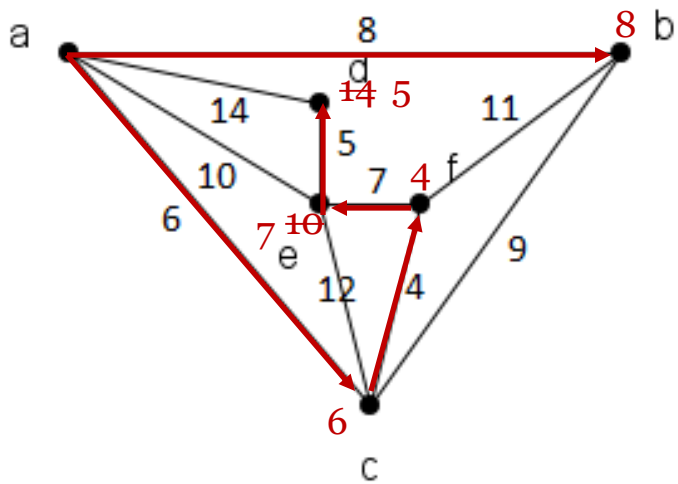


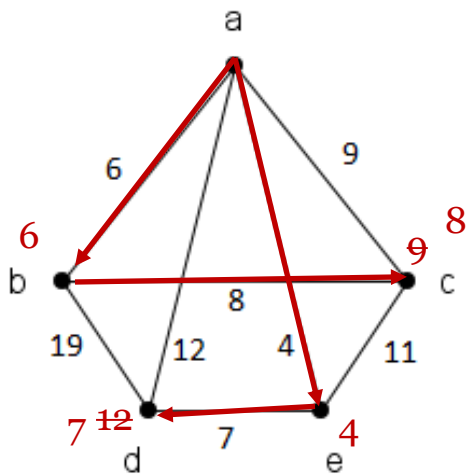
Prim's algorithm

- For each graph below, compute the tree produced by Prim's and Dijkstra's algorithms, starting at *a*



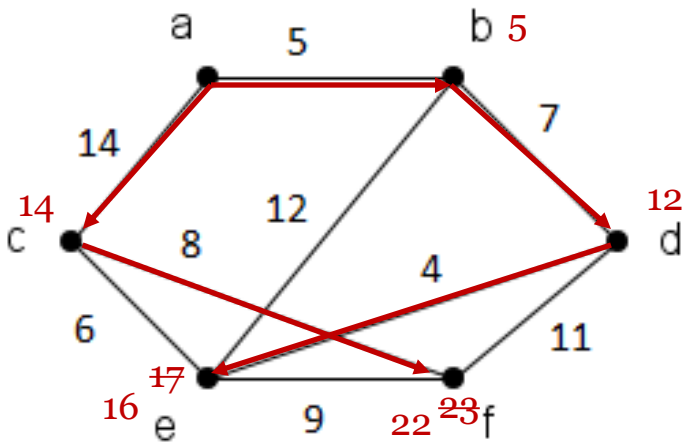


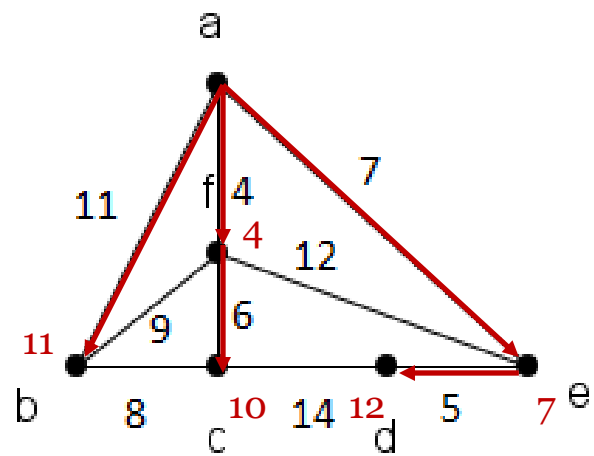


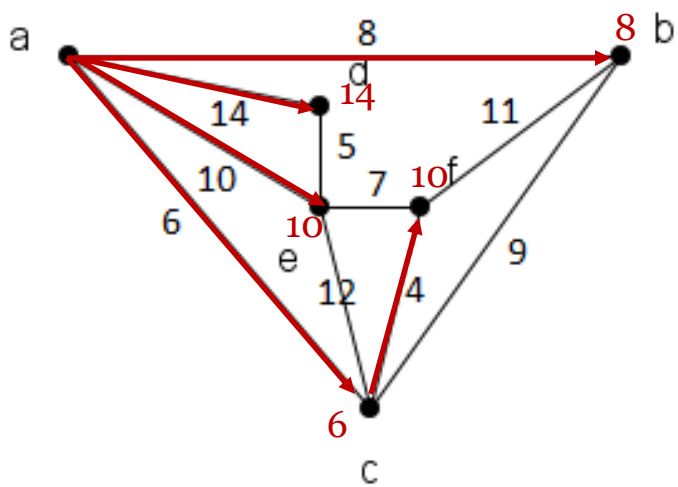


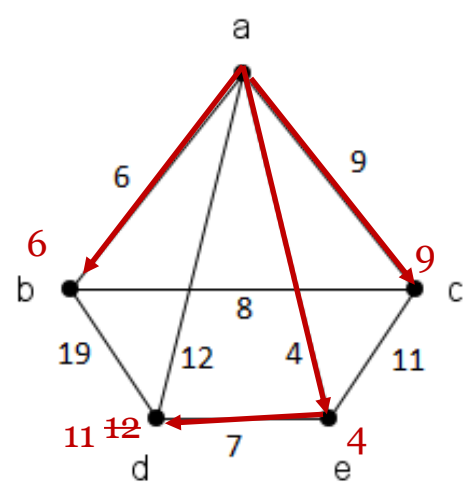
Dijkstra's algorithm

- For each graph below, compute the tree produced by Prim's and Dijkstra's algorithms, starting at *a*



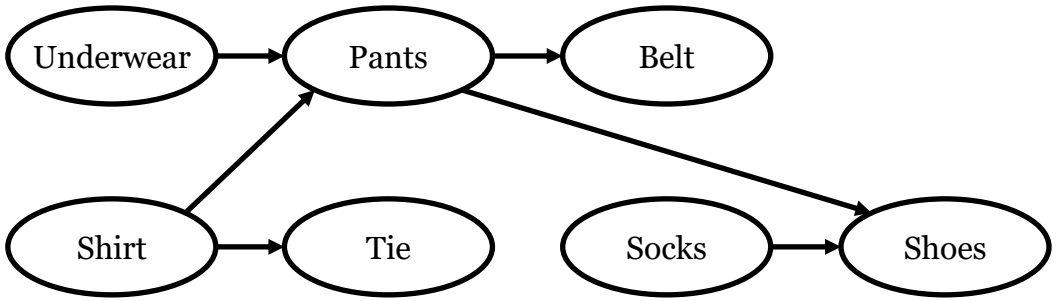






Directed acyclic graphs

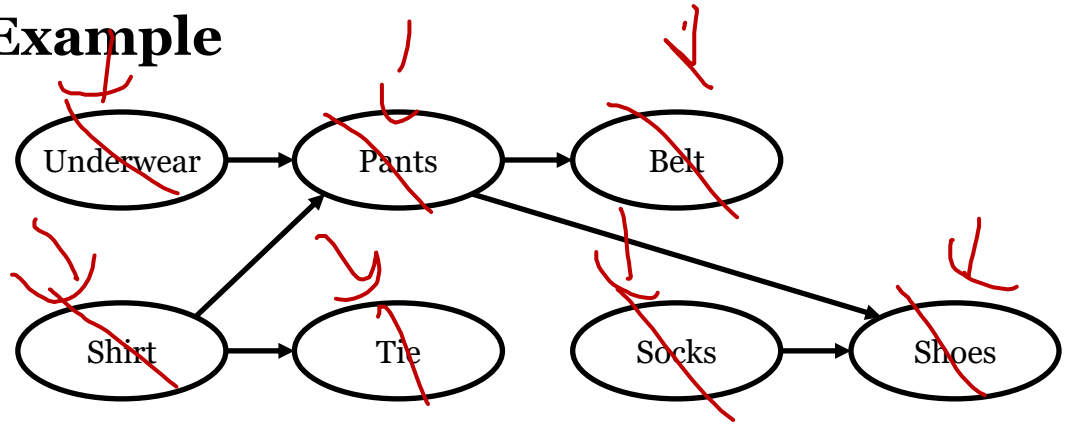
- Directed graphs with no cycles
- Sometimes used to represent processes or requirements
- **Example**



- *Important question*
  - How can I finish everything while meeting all requirements?

Topological sorting

- **Input:** directed acyclic graph (DAG)
- **Output:** vertices ordered so that all edges point right
- **Example**

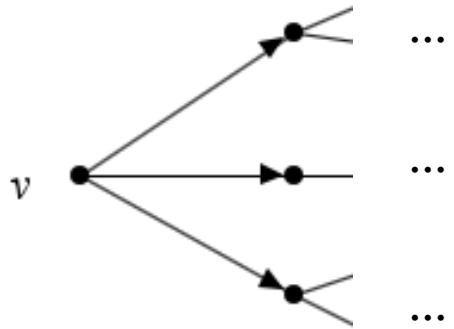


Socks
Shirt
Tie
Underwear
Pants
Shoes
Belt

- **Algorithm**
  - Perform DFS
  - Add all vertices to stack when you mark them as explored
  - Return vertices in reverse order
- **Example (above)**
  - DFS: belt, shoes, pants, underwear, tie, shirt, socks
  - Reversed: socks, shirt, tie, underwear, pants, shoes, belt

Topological sorting correctness

- *Proof sketch*
  - Everything that depends on v is explored before v is finished

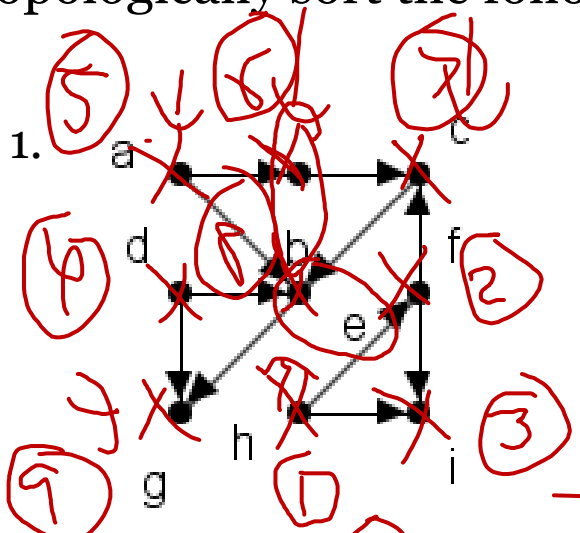


  - In reverse, v appears before everything that depends on it

Topological sorting exercise

Topologically sort the following DAGs:

1.

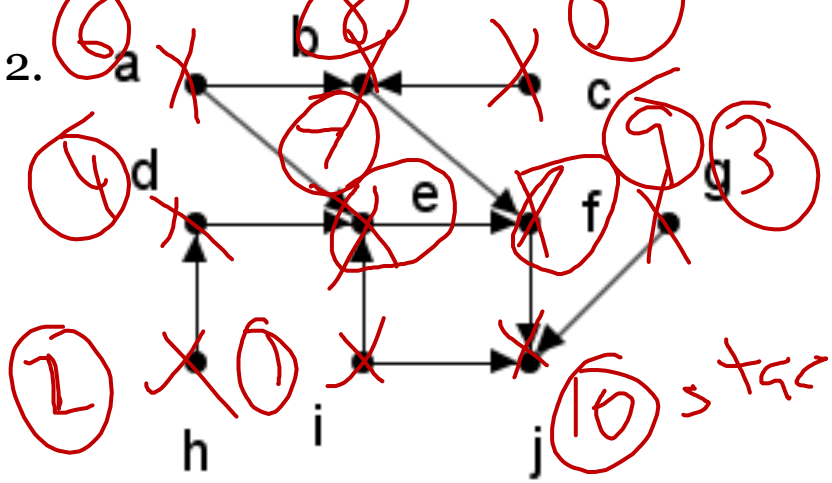


stack:

d
a
b
c
e
g

topo order.
h f i d a b c e g

2.



stack:

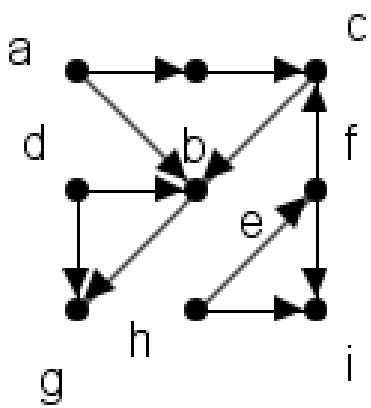
c
e
b
f
j

topo order
i h g d c a e b f j

Topological sorting exercise

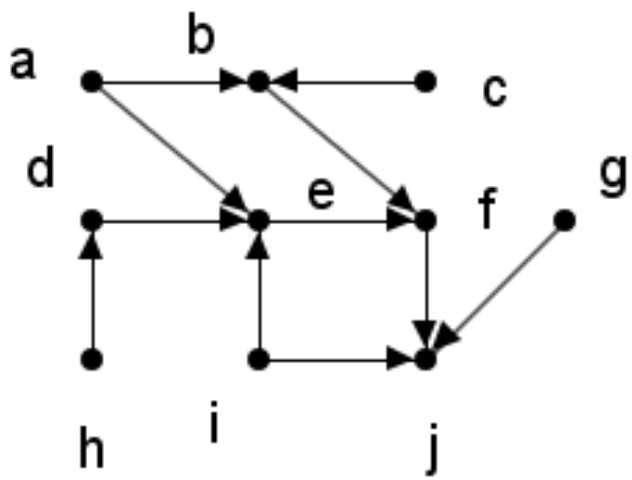
Topologically sort the following DAGs:

1.



DFS: g, e, c, b, a, d, i, f, h
Sorted: h, f, i, d, a, b, c, e, g

2.



DFS: j, f, b, e, a, c, d, g, h, i
Sorted: i, h, g, d, c, a, e, b, f ,j

Coming up

- Weighted graph algorithms
- Backtracking

- **Recommended readings:** Sections 13.1-13.3, 14.1-14.2, and 15.1-15.3
- *Practice problems:* R-13.1, R-13.2, R-13.5, R-13.7, C-13.4, R-14.2, R-14.3, C-14.3, C-14.4, A-14.6, R-15.9, C-15.1, C-15.4, A-15.6